# path_finder code map

What to read to understand the shipped code, one line per function, and where to split the big files. Generated 2026-10-21 from the committed tree (368 functions across the shipped files).

## Read these, in this order

| Read | Why in this order |
|---|---|
| `pyproject.toml` | Three lines matter: the build backend, the vendored files, the console script. Start here to see what ships. |
| `path_finder/cli.py`, `repo.py`, `window.py` | Small. Shows how the command finds the code (checkout or vendored copy) and launches the window. |
| `discovery/discover.py` | The data layer the watcher depends on: data directory rule, registry, identity files, sessions.json. |
| `viz/watcher.py` | The core. Transcript tailing, the message state machine, the SPEC parser, the HTTP and SSE server. Largest Python file. |
| `viz/web.html` | The window you use. Largest file overall; most of the refactor value is here. |
| `viz/index.html` | The full-width page. Same data, simpler rendering. Read last or skip if you only keep the window. |
| `plugin/skills/*/SKILL.md` | Two short instruction files, no code. Read once. |

## Skip these

| Skip | What it is |
|---|---|
| `tests/` | Unit and integration tests, sanitized transcript fixtures, the Node render harness. Useful later as a safety net for the refactor, not for understanding the code. |
| `handoff/` | Builder to validator handoffs and the manager's status log. Team process only. |
| `docs/` | Format references, the plan, the UI frames, this guide. Background, not code. |
| `.claude/roles/`, `CLAUDE.md`, `SPEC.md` | How the team sessions behave and the task list. Not shipped. The watcher reads a SPEC.md from the watched folder only to fill the task panel. |
| `.claude-plugin/marketplace.json`, `plugin/.claude-plugin/plugin.json`, `plugin/README.md` | Manifests and install notes. Glance once, nothing to understand. |
| `path_finder/__init__.py`, `README.md`, `.gitignore` | Empty or informational. |

## pyproject.toml in three lines

- Build backend is hatchling (requires hatchling>=1.21, build-backend hatchling.build); package path-finder 0.2.0, Python >=3.9, no dependencies.

- The wheel packages path_finder and force-includes discovery/discover.py, viz/watcher.py, viz/web.html, and viz/index.html under path_finder/_vendor/ so the CLI works without a checkout.
- Console script path-finder maps to path_finder.cli:main; the sdist additionally includes those four files, pyproject.toml, and README.md.

## How to approach the refactor

The pattern is the same in every big file: one file holds several concerns that only touch at a few well-defined points. Split along those points, keep the public entry points where they are, and let the tests prove nothing moved.

1. **Keep the entry points stable.** `path-finder` runs `discover.py` and `watcher.py` as scripts, the plugin calls the command, and the pages fetch `/state` and `/events`. Turn each big file into a small entry file that imports from a package, so nothing outside changes.
2. **Move pure logic first.** Parsing and state transitions have no I/O and are already covered by unit tests. They move with the least risk.
3. **Then the I/O edges.** Tailing, the HTTP server, the event log, the browser launch. Each becomes its own module with one job.
4. **The page last.** Pull the CSS into its own file, then split the script into the files suggested below, loaded in order by the page. The Node harness renders the page, so it catches a broken load order.
5. **Run the suite after every move.** 423 tests pass today. A move that changes a count is a move that changed behaviour.

## 1. `path_finder/cli.py` 96 lines, 8 functions

### checkout resolution

| Function | Line | What it does |
|---|---|---|
| `find_code_folder` | 17 | Returns the checkout, or the vendored _vendor folder when no checkout is found. |

### argument defaults

| Function | Line | What it does |
|---|---|---|
| `has_root_flag` | 26 | Checks whether --root or --root= is already among the arguments. |
| `with_default_root` | 30 | Appends --root <cwd> for list, done, serve, and map when none was given. |

### script dispatch

| Function | Line | What it does |
|---|---|---|
| `run_script_in_place` | 36 | Sets sys.argv and runs discover.py or watcher.py as __main__ via runpy. |
| `dispatch` | 66 | Routes a command to discover.py, watcher.py, or open_window; unknown commands print usage. |

### CLI commands

| Function | Line | What it does |
|---|---|---|
| `usage` | 42 | Builds the help text listing commands, root defaulting, and the data directory rule. |
| `parse_window_arguments` | 58 | Parses --port, --page, and --root for the window subcommand. |
| `main` | 80 | Entry point: handles help, dispatches, and maps RepoNotFound and SystemExit to exit codes. |

### Observations

- Already small enough; the only candidate change is moving find_code_folder into repo.py beside data_dir_description, which repeats the same fallback.
- run_script_in_place mutates sys.argv and never restores it, fine for a one-shot CLI.

## 2. `path_finder/repo.py` 68 lines, 9 functions

### checkout resolution

| Function | Line | What it does |
|---|---|---|
| `RepoNotFound` | 10 | Exception raised when no folder with discovery/discover.py and viz/watcher.py is found. |
| `is_repo` | 14 | True when the folder holds both required script files. |

| | | |
|---|---|---|
| `candidate_folders` | 18 | Yields PATH_FINDER_REPO, the package's parent, the cwd, and every cwd ancestor. |
| `find_repo` | 28 | Returns the first candidate folder that is a repo, else raises RepoNotFound. |

#### script paths

| Function | Line | What it does |
|---|---|---|
| `discover_script` | 38 | Returns <folder>/discovery/discover.py. |
| `watcher_script` | 42 | Returns <folder>/viz/watcher.py. |

#### vendored fallback

| Function | Line | What it does |
|---|---|---|
| `vendored_copies_present` | 46 | True when the package's _vendor folder contains both scripts. |

#### data directory rule

| Function | Line | What it does |
|---|---|---|
| `data_dir` | 50 | PATH_FINDER_HOME, else <checkout>/discovery when not vendored and pyproject.toml exists, else ~/.path_finder. |
| `data_dir_description` | 59 | Formats a two-line help string naming the code folder and the data directory. |

#### Observations

- Already small enough; I would only pull find_code_folder from cli.py into it so the checkout-or-vendor fallback lives in one place.
- data_dir here and data_dir in discover.py implement the same rule twice with slightly different checkout tests.

### 3. `path_finder/window.py` 98 lines, 7 functions

#### window launch

| Function | Line | What it does |
|---|---|---|
| `page_url` | 29 | Builds http://127.0.0.1:<port>/<page>, mapping index aliases to the root path. |
| `open_app_window` | 77 | Opens the URL in a 380x720 --app browser window, or the default browser when none is found. |
| `open_window` | 91 | Starts the watcher if it is not answering, then opens the page in an app-mode window. |

#### server lifecycle

| Function | Line | What it does |
|---|---|---|
| `server_answers` | 34 | True when GET /state on the port succeeds within one second. |
| `server_log_path` | 42 | Returns the temp-dir log file path for a serve process on that port. |

| | | |
|---|---|---|
| `start_server_in_background` | 46 | Spawns watcher.py serve detached, logs to file, waits up to 8 s for /state, else raises. |

### browser detection

| Function | Line | What it does |
|---|---|---|
| `app_mode_browser` | 63 | Finds a Chrome, Chromium, Brave, or Edge binary in macOS app bundles or on PATH. |

### Observations

- Already small enough; server lifecycle (server_answers, server_log_path, start_server_in_background) could become server.py if more commands need it.
- start_server_in_background returns the Popen handle but open_window only prints its pid; the process is intentionally left running.

## 4. `discovery/discover.py` 279 lines, 27 functions

### data directory rule

| Function | Line | What it does |
|---|---|---|
| `checkout_holding` | 15 | Walks up from a path to the first ancestor that has discovery/, viz/, and pyproject.toml. |
| `data_dir` | 22 | Picks the data folder: PATH_FINDER_HOME, else the checkout's discovery/, else ~/.path_finder. |

### time helpers

| Function | Line | What it does |
|---|---|---|
| `now_iso` | 37 | Returns the current UTC time as an ISO-8601 string with a Z suffix. |

### process liveness

| Function | Line | What it does |
|---|---|---|
| `pid_is_alive` | 41 | Reports whether a PID still exists, treating permission errors as alive. |
| `parent_and_command` | 51 | Asks ps for a PID's parent PID and command name, or (None, None). |
| `find_claude_ancestor_pid` | 65 | Climbs the parent chain up to eight levels to find the enclosing claude process. |

### registry reading

| Function | Line | What it does |
|---|---|---|
| `socket_address` | 79 | Builds the uds:/tmp/cc-socks/<pid>.sock address for a session PID. |
| `read_registry_entry` | 83 | Loads one ~/.claude/sessions JSON file into a normalised dict with pid, name, cwd, socket, session_id. |
| `live_sessions` | 100 | Reads every registry file and keeps the entries whose PID is still alive. |

| | | |
|---|---|---|
| `under_root` | 109 | Tells whether a session cwd sits under the given root; no root means always true. |
| `own_session` | 117 | Finds the calling session's registry entry via its claude ancestor PID. |

### identity files

| Function | Line | What it does |
|---|---|---|
| `identity_path` | 124 | Returns the identity/<pid>.json path for a session PID. |
| `read_identity` | 128 | Loads a session's identity JSON, or None when missing or malformed. |
| `write_identity` | 136 | Writes the session entry plus role, label, and timestamp to its identity file. |
| `drop_identities_of_dead_sessions` | 145 | Deletes identity files whose PID is not in the live set or is not numeric. |

### sessions.json rebuild

| Function | Line | What it does |
|---|---|---|
| `with_identity` | 152 | Merges a session entry with its identity's role, label, and identified_at. |
| `is_identified` | 162 | True when a directory entry carries a role. |
| `directory_entries` | 166 | Lists live sessions, prunes dead identities, and attaches identities to each entry. |
| `lock_directory` | 172 | Takes an exclusive flock on sessions.json.lock and returns the open handle. |
| `write_directory` | 179 | Atomically writes updated_at and the entries to sessions.json via a temp file. |
| `rebuild_directory` | 188 | Under the lock, recomputes the directory entries, writes sessions.json, and returns them. |

### CLI commands

| Function | Line | What it does |
|---|---|---|
| `command_identify` | 198 | Stores the given role for the calling session, rebuilds the directory, prints the identity. |
| `command_self` | 213 | Prints the calling session with its role and label; exits 2 when not identified. |
| `command_list` | 226 | Rebuilds the directory and prints one JSON row per live session under --root. |
| `command_done` | 239 | Exits 1 naming unidentified sessions under --root, else 0 with a success line. |
| `build_parser` | 252 | Builds the argparse parser with the identify, self, list, and done subcommands. |
| `main` | 273 | Parses argv and runs the chosen subcommand's function. |

### Suggested split

| New module | Purpose | Takes these groups |
|---|---|---|
| `discovery/paths.py` | Resolves where discovery data lives and the derived sessions.json and identity/ paths. | data directory rule, time helpers |
| `discovery/processes.py` | Process inspection: PID liveness, ps parent lookup, and finding the enclosing claude process. | process liveness |
| `discovery/registry.py` | Reads ~/.claude/sessions entries, filters live ones, and finds the caller's own session. | registry reading |
| `discovery/identity.py` | Reads, writes, and prunes identity/<pid>.json files and rebuilds sessions.json under a lock. | identity files, sessions.json rebuild |
| `discovery/discover.py` | Thin CLI entry: argparse wiring and the four command functions over the modules above. | CLI commands |

### Observations

- Module-level DISCOVERY_DIR, DIRECTORY_PATH, and IDENTITY_DIR are computed at import; a split must keep that in paths.py or pass them in.
- data_dir duplicates path_finder/repo.py data_dir logic with a different checkout test (checkout_holding vs pyproject.toml beside code).
- The wheel vendors this file at path_finder/_vendor/discovery/discover.py; checkout_holding then fails and ~/.path_finder is used, which is intended.
- with_identity reads identified_at or the legacy set_at key, a hint of an older identity format.
- The file is run in place via runpy by cli.py, so a split into a package must keep discover.py runnable as a script with sibling imports resolvable.

## 5. `viz/watcher.py` 1240 lines, 106 functions

### paths and config

| Function | Line | What it does |
|---|---|---|
| `is_checkout` | 24 | True when a folder holds discovery/, viz/, and pyproject.toml together. |
| `checkout_root` | 28 | Walks up from a folder to the first checkout, or None. |
| `resolve_data_dir` | 35 | Picks the data directory: PATH_FINDER_HOME, else the checkout's discovery/, else ~/.path_finder. |

### registry and sessions

| Function | Line | What it does |
|---|---|---|
| `pid_is_alive` | 92 | Signals PID 0 to learn whether the process still exists. |
| `socket_address` | 102 | Builds the uds:/tmp/cc-socks/<pid>.sock address for a PID. |
| `encoded_cwd` | 106 | Turns a folder path into the transcript folder name by replacing non-alphanumerics with dashes. |
| `read_registry_entry` | 110 | Reads one ~/.claude/sessions file into pid, name, cwd, socket, session id, start time. |

| | | |
|---|---|---|
| `read_identity` | 128 | Reads a session's role and label from its identity file, or Nones. |
| `live_sessions` | 137 | Lists every registry entry whose PID is still alive. |
| `under_root` | 146 | True when a session's cwd sits under the watched root folder. |
| `identified_live_sessions` | 754 | Live sessions under the root, each merged with its identity role and label. |

### transcript lookup

| Function | Line | What it does |
|---|---|---|
| `transcripts_named_by_session_id` | 154 | Finds transcript files named after the session id, preferring the session's own cwd folder. |
| `transcripts_started_after_session` | 163 | Fallback: transcripts in the cwd folder modified since the session started. |
| `resolve_transcript` | 175 | Maps a registry entry to its transcript path and records which method found it. |
| `session_map` | 1177 | Live sessions under the root with identity and resolved transcript, for map and state. |

### entry helpers

| Function | Line | What it does |
|---|---|---|
| `pid_from_socket` | 193 | Extracts the PID from a socket address string, or None. |
| `bare_name` | 200 | Strips the trailing [ref] suffix from a display name. |
| `first_quoted` | 206 | Returns the first double-quoted string in a text, used for idle notices. |
| `content_text` | 211 | Flattens a message's content, string or text blocks, into one string. |
| `tool_result_blocks` | 220 | Returns the tool_result blocks of a user entry. |
| `send_result_from` | 227 | Gets the SendMessage result dict from toolUseResult or by parsing the block's JSON text. |
| `entry_source` | 263 | Names the kind of transcript entry behind a change, for the page caption. |

### activity line

| Function | Line | What it does |
|---|---|---|
| `first_line` | 247 | First non-empty line of a text, truncated to the activity limit. |
| `describe_tool_use` | 252 | One-line activity for a tool call: its description, else tool name plus its main target. |
| `StateModel.set_activity` | 461 | Stores a new activity line and its timestamp when it changed. |

### state model

| Function | Line | What it does |
|---|---|---|
| `StateModel` | 290 | In-memory sessions and messages, updated one transcript entry at a time. |

| | | |
|---|---|---|
| `StateModel.__init__` | 291 | Creates the empty maps: sessions, messages, tool-use keys, names, unmatched enqueues, watches, sources. |
| `StateModel.register_session` | 301 | Creates or updates a session record from a registry row, with default fields. |
| `StateModel.session` | 330 | Returns the session for a PID, registering a bare one if unknown. |
| `StateModel.resolve_target` | 333 | Turns a message target, socket address or display name, into a PID. |
| `StateModel.learn_name` | 339 | Remembers that a display name belongs to a PID. |
| `StateModel.snapshot` | 345 | Returns sessions, messages sorted by time, and the latest source per PID. |
| `StateModel.note_source` | 352 | Records which entry last changed a session, with a sequence number. |
| `StateModel.apply` | 356 | Dispatches one entry to its handler by type, refreshes waiting, returns the list of changes. |

### waiting

| Function | Line | What it does |
|---|---|---|
| `StateModel.waiting_targets` | 381 | PIDs a session waits on: open watches plus unanswered sent messages, in time order. |
| `StateModel.refresh_waiting` | 396 | Recomputes waiting_on for every session and marks the ones that changed. |
| `StateModel.add_watch` | 502 | Adds a watched PID or name to the session's watching list. |
| `StateModel.watched_pid_of` | 510 | Resolves a watching entry, PID or name, to a PID. |
| `StateModel.clear_watch` | 513 | Removes every watching entry that points at the same session as the given one. |
| `StateModel.apply_idle_notice` | 691 | Idle notice: records a one-shot idle_notice message, clears the watch, marks the receiver WORKING. |

### session status

| Function | Line | What it does |
|---|---|---|
| `StateModel.set_status` | 403 | Sets WORKING or IDLE on a session if it differs. |
| `StateModel.apply_agent_name` | 433 | Handles an agent-name entry: updates the session's display name. |
| `StateModel.apply_assistant` | 444 | Assistant entry: marks WORKING, records last text and activity, hands SendMessage calls to apply_send. |
| `StateModel.apply_system` | 718 | turn_duration entry: marks IDLE, records idle_since, activity falls back to last assistant text. |

### message state machine

| Function | Line | What it does |
|---|---|---|
| `StateModel.new_message` | 409 | Creates a message record with every field defaulted, then applies overrides. |
| `StateModel.apply_send` | 468 | A SendMessage call: pure subscription becomes a watch, otherwise a PENDING message keyed by tool use. |

| | | |
|---|---|---|
| `StateModel.apply_user` | 524 | User entry: routes peer deliveries, idle notices, delivery notices, tool results, or typed input. |
| `StateModel.apply_send_result` | 544 | SendMessage result: rekeys the message by msg_id and moves it to IN_INBOX, or marks it FAILED. |
| `StateModel.claim_unmatched_enqueue` | 581 | Gives a message an enqueue timestamp seen earlier on the receiver before the sender's side was read. |
| `StateModel.apply_queue_operation` | 588 | Queue entry: enqueue raises the inbox and classifies the content; dequeue or remove lowers it. |
| `StateModel.apply_peer_enqueue` | 605 | A peer message landing in an inbox: matches the sender's pending message or parks the enqueue. |
| `StateModel.apply_peer_delivery` | 628 | The receiver read a peer message: stage READ, read_at set, receiver marked WORKING. |
| `StateModel.adopt_pending_send` | 650 | Rekeys the sender's oldest unmatched pending message to the msg_id seen on the receiver. |
| `StateModel.apply_attachment` | 667 | Queued-command attachment: treats a mid-turn peer delivery or notice as if it were a user entry. |
| `StateModel.apply_delivery_notice` | 682 | A NOT delivered notice: marks the named msg_ids FAILED with the reason. |

### utilities

| Function | Line | What it does |
|---|---|---|
| `now_iso` | 728 | Current UTC time as an ISO string with milliseconds and a Z. |
| `parse_timestamp` | 1057 | Parses an ISO timestamp with milliseconds and Z into an aware datetime. |

### SSE events

| Function | Line | What it does |
|---|---|---|
| `EventBroadcaster` | 732 | Fan-out of change dicts to every open /events subscriber. |
| `EventBroadcaster.__init__` | 733 | Empty subscriber set under a lock. |
| `EventBroadcaster.subscribe` | 737 | Creates a queue for one client and registers it. |
| `EventBroadcaster.unsubscribe` | 743 | Drops a client's queue. |
| `EventBroadcaster.publish` | 747 | Puts a change on every subscriber's queue. |
| `VizRequestHandler.send_events` | 968 | Streams a snapshot then every change as server-sent events, with keepalives. |
| `VizRequestHandler.write_sse` | 989 | Writes one data: line in SSE format and flushes. |

### SPEC task parsing

| Function | Line | What it does |
|---|---|---|
| `task_title` | 762 | The task text up to its first period. |
| `task_area` | 768 | Maps a task ID's letter to Discovery, Visualization, Packaging, or the letter. |

| | | |
|---|---|---|
| `parse_spec_tasks` | 773 | Reads the Tasks section of SPEC.md into id, state, title, text, owner, area records. |
| `task_counts` | 797 | Counts tasks by done, in_progress, todo. |
| `SpecTasks` | 801 | Re-reads SPEC.md only when its size or mtime changes. |
| `SpecTasks.__init__` | 802 | Stores the path with no signature and no tasks yet. |
| `SpecTasks.refresh` | 807 | Re-parses when the file signature changed; returns whether the task list changed. |
| `SpecTasks.payload` | 826 | The tasks, their counts, and the spec path, for /state and the tasks event. |
| `Watcher.refresh_tasks` | 900 | Emits a tasks event when SPEC.md changed. |

### polling loop

| Function | Line | What it does |
|---|---|---|
| `Watcher` | 830 | Owns the tailers, the state model, the SPEC reader, the broadcaster, and the event log. |
| `Watcher.__init__` | 831 | Wires root, fixtures, spec path, log path, lock, and stop event together. |
| `Watcher.close` | 853 | Stops the loop and closes the log file. |
| `Watcher.session_rows` | 859 | Rows to tail: fixture PIDs with their files, or live sessions with resolved transcripts. |
| `Watcher.refresh_sessions` | 871 | Registers new sessions and tailers, emits session changes, drops sessions that left. |
| `Watcher.poll_once` | 892 | One tick: refresh sessions and tasks, apply new entries in time order, emit changes. |
| `Watcher.entries_in_time_order` | 904 | Polls every tailer and merges the new entries sorted by timestamp. |
| `Watcher.run` | 915 | Loops poll_once every 250 ms until stopped. |

### event log

| Function | Line | What it does |
|---|---|---|
| `Watcher.emit` | 844 | Appends a change to events.jsonl and publishes it to subscribers. |

### HTTP server

| Function | Line | What it does |
|---|---|---|
| `Watcher.snapshot` | 920 | The full /state payload: time, data dir, model snapshot, tasks. |
| `VizRequestHandler` | 925 | HTTP handler serving /state, /events, /web, and the static page files. |
| `VizRequestHandler.__init__` | 929 | Static file handler rooted at the viz folder. |
| `VizRequestHandler.log_message` | 932 | Silences request logging unless verbose. |
| `VizRequestHandler.do_GET` | 936 | Routes /state, /events, page aliases, else serves a static file. |
| `VizRequestHandler.static_dir_for` | 949 | Finds which static folder holds the requested file. |

| | | |
|---|---|---|
| `VizRequestHandler.send_json` | 956 | Writes a JSON response with no-store caching. |
| `VizRequestHandler.send_state` | 965 | Sends the watcher snapshot as JSON. |

### CLI commands

| Function | Line | What it does |
|---|---|---|
| `command_serve` | 994 | serve: builds the watcher, binds the port, runs the poller thread and the HTTP server. |
| `entries_in_time_order` | 1016 | Module-level twin of the method: merges whole fixture files by timestamp for replay. |
| `command_replay` | 1033 | replay: writes fixture lines into fresh per-PID files at a chosen speed, with until and after cuts. |
| `parse_fixture_argument` | 1061 | Parses a PID=PATH argument. |
| `command_state` | 1068 | state: feeds fixture files through a fresh model and prints the snapshot or the change stream. |
| `command_tail` | 1157 | tail: prints kept entries of one transcript, optionally following it, with stats at exit. |
| `command_map` | 1187 | map: prints pid, session id, transcript, label, and method; exit 1 if any is unresolved. |
| `build_parser` | 1197 | Defines the map, tail, state, serve, and replay subcommands and their flags. |
| `main` | 1234 | Parses arguments and exits with the chosen command's return code. |

### transcript tailing

| Function | Line | What it does |
|---|---|---|
| `entry_is_kept` | 1086 | The keep rules: known type, not sidechain, meta only for peer or system turns, kept attachment types. |
| `TranscriptTailer` | 1099 | Reads one transcript by byte offset, buffering partial lines and counting skips. |
| `TranscriptTailer.__init__` | 1100 | Starts at byte 0 or at the current end, with empty buffer and counters. |
| `TranscriptTailer.read_new_bytes` | 1108 | Reads bytes appended since the last offset; restarts if the file shrank. |
| `TranscriptTailer.complete_lines` | 1124 | Splits a chunk into whole lines and keeps the trailing partial line for next time. |
| `TranscriptTailer.parse_line` | 1130 | Parses one line to a dict and applies the keep rules; counts malformed and unknown. |
| `TranscriptTailer.poll` | 1148 | New kept entries since the last poll. |

### Suggested split

| New module | Purpose | Takes these groups |
|---|---|---|

| | | |
|---|---|---|
| `viz/paths.py` | Constants and the data-directory rule, shared with discover.py. | paths and config |
| `viz/sessions.py` | Registry reading, liveness, identity, and transcript lookup. | registry and sessions, transcript lookup |
| `viz/tailer.py` | TranscriptTailer and the keep rules. | transcript tailing |
| `viz/entries.py` | Pure helpers that read fields out of a transcript entry, plus the activity line. | entry helpers, activity line |
| `viz/state.py` | StateModel: sessions, the message state machine, waiting, status. | state model, message state machine, waiting, session status |
| `viz/tasks.py` | SPEC.md parsing and the SpecTasks cache. | SPEC task parsing |
| `viz/server.py` | Watcher loop, EventBroadcaster, event log, HTTP and SSE handler. | polling loop, event log, SSE events, HTTP server |
| `viz/watcher.py` | Thin entry: argparse and the five commands, importing the rest. | CLI commands, utilities |

### Observations

- StateModel is 430 lines and holds four concerns: session status, the message state machine, watches and waiting, and the activity line. The message handlers (apply_send, apply_send_result, apply_peer_enqueue, apply_peer_delivery, adopt_pending_send, claim_unmatched_enqueue) are the hard part and deserve their own module with the stage diagram at the top.
- entries_in_time_order exists twice: as a Watcher method over tailers and as a module function over whole fixture files. One ordering helper that takes (timestamp, pid, index) tuples would serve both.
- Module constants DATA_DIR, IDENTITY_DIR, SPEC_PATH, and EVENT_LOG_PATH are computed at import time from the file's own location; a function that takes the data dir as an argument would make the model testable without environment variables.
- Three message keys exist for one message over its life: pending:<tool_use_id>, then msg_id, and idle:<uuid> for notices. Naming the key scheme in one place would make apply_send_result and adopt_pending_send easier to follow.
- The CLI commands (serve, replay, state, tail, map) are interleaved with the classes they use; grouping them at the bottom or in their own module makes the read order top-down.

## 6. `viz/web.html` 1542 lines, 160 functions

Inline CSS: about 120 lines.

### utilities

| Function | Line | What it does |
|---|---|---|
| `parseMs` | 237 | Parses an ISO timestamp to epoch milliseconds, returning 0 when missing or invalid. |
| `labelOf` | 255 | Picks a display label for a session: label, role, name, or pid. |
| `pidFromSocket` | 259 | Extracts the PID number from a uds socket address string, else null. |
| `labelFor` | 264 | Resolves a pid, socket address, or name to a session label, falling back to the raw value. |

| | | |
|---|---|---|
| `colorOf` | 273 | Chooses a node color by role keyword, else a palette color by pid order. |
| `folderOf` | 282 | Returns the last path segment of a working directory. |
| `pidByName` | 287 | Finds a session pid whose name or label matches the given string. |
| `truncate` | 307 | Collapses whitespace and trims text to a maximum length with an ellipsis. |
| `el` | 673 | Creates an HTML element with optional class and text. |
| `svgEl` | 690 | Creates an SVG element with the given attributes. |

### state and fetch

| Function | Line | What it does |
|---|---|---|
| `advanceFeedClock` | 242 | Moves the feed clock forward when a newer feed timestamp arrives, noting wall time. |
| `feedNowMs` | 250 | Returns the current feed time extrapolated from the newest feed timestamp seen. |
| `applySnapshot` | 322 | Replaces all state from a full snapshot and derives the initial caption. |
| `newestMessage sort` | 334 | Inline comparator picking the most recent message for the snapshot caption. |
| `applyChange` | 340 | Applies one SSE change event to sessions, messages, tasks, sources, and the feed clock. |
| `loadState` | 1499 | Fetches the full state snapshot and schedules a render, even on failure. |

### message queries

| Function | Line | What it does |
|---|---|---|
| `endpointsOf` | 292 | Returns [fromPid, toPid] for a message, resolving a missing to_pid by name. |
| `messageMs` | 299 | Returns the message's latest lifecycle timestamp (read, accepted, enqueued, or sent) in ms. |
| `sentMs` | 303 | Returns the message send time in ms, falling back to read time. |
| `messagesOnly` | 403 | Returns messages of kind message that have resolvable endpoints. |
| `noticesOnly` | 407 | Returns stored idle_notice records. |
| `latestSendOf` | 411 | Finds the most recent message sent by a pid. |
| `statusText` | 420 | Returns Idle, Sending, or Working for a session based on its latest send stage. |
| `failedIsResolved` | 426 | True when a later message on the same pair reached inbox or read after a failure. |
| `replyAfterRead` | 434 | Finds the earliest reverse-direction message sent after this message was read. |
| `withinHorizon` | 454 | True when the message is within the ten-minute ring window. |
| `visibleArrows` | 458 | Filters messages to those that should currently draw an arrow. |

| | | |
|---|---|---|
| `visibleArrows filter` | 459 | Inline predicate deciding visibility per stage: failed, pending, inbox, or fading read. |
| `pulsingPids` | 467 | Collects pids that received an idle notice within the pulse window. |

### caption

| Function | Line | What it does |
|---|---|---|
| `noteCaption` | 312 | Updates the caption entry and pair text from an incoming change event. |
| `renderTitle` | 1460 | Writes the title bar summary: folder, session count, working count, in-flight count. |

### task panel

| Function | Line | What it does |
|---|---|---|
| `applyTasks` | 369 | Stores the task list and task counts from a snapshot or tasks event. |
| `ownerPidOf` | 375 | Finds the session pid whose role, label, or name contains the task owner string. |
| `taskedPids` | 385 | Maps each session pid to the ids of in-progress tasks it owns. |
| `taskBadge` | 397 | Builds a small badge element listing in-progress task ids. |
| `countsText` | 1364 | Formats done, in progress, and to do counts for the panel tab. |
| `areaOf` | 1369 | Returns a task's area name, inferred from its id prefix when absent. |
| `folderCountsText` | 1375 | Formats nonzero per-state counts for one task folder. |
| `foldersOf` | 1384 | Groups tasks by area into a Map. |
| `toggleFolder` | 1398 | Flips a folder's collapsed flag in localStorage and re-renders. |
| `togglePanel` | 1404 | Flips the panel collapsed flag, persists it, and re-renders. |
| `toggleTask` | 1410 | Opens or closes a task's full text and re-renders. |
| `renderPanel` | 1415 | Rebuilds the task panel tab and folder list with task cards. |
| `folder head click` | 1432 | Toggles the clicked folder. |
| `taskEl` | 1440 | Builds one task card with id, title, state, owner dot, and expandable text. |
| `task click` | 1454 | Toggles the clicked task open. |
| `task mouseenter` | 1455 | Records the hovered task id and adds the hover class. |
| `task mouseleave` | 1456 | Clears the hovered task id and removes the hover class. |
| `panelTab click` | 1534 | Wires the panel tab to togglePanel. |

### arrows and curves

| Function | Line | What it does |
|---|---|---|
| `readOpacity` | 444 | Computes arrow opacity for a read message: flash, mute, then fade when replied or expired. |
| `exitPoint` | 666 | Finds where a line from a node center toward a point leaves the node rectangle plus gap. |

| | | |
|---|---|---|
| `hasReversedPair` | 801 | True when any message has a counterpart in the opposite direction. |
| `reversed predicate` | 803 | Inline test for whether the reverse arrow key exists. |
| `arrowKey` | 806 | Returns the from>to key for a message's endpoints. |
| `controlCandidates` | 885 | Lists candidate bezier control points: pod lifts or pulls toward the ring center. |
| `renderArrows` | 945 | Groups messages per pair, places curves and labels, and creates or updates path and label elements. |
| `newest reduce` | 954 | Inline reducer picking the most recently sent message of a pair. |
| `fading` | 961 | True when a read message has a reply and is fading out. |
| `entries sort` | 966 | Inline comparator ordering arrows newest first, then by key. |
| `path mouseenter` | 981 | Lights the hovered arrow and re-renders. |
| `path mouseleave` | 982 | Clears the lit arrow and re-renders. |

### ring layout and geometry

| Function | Line | What it does |
|---|---|---|
| `ringSessions` | 476 | Returns all sessions sorted by pid for ring placement. |
| `layoutNodes` | 482 | Resets the layout record with graph width, compact or pod mode, and card size. |
| `estimateNodeHeight` | 491 | Estimates a node's height from its activity text when it is not yet measured. |
| `measureNodes` | 498 | Records each node's measured or estimated size and the tallest node height. |
| `sizeOf` | 507 | Returns a node's [width, height], defaulting to the layout card size. |
| `ringPositions` | 511 | Places sessions evenly on an ellipse of radii rx and ry starting at the top. |
| `overlappingPairs` | 520 | Lists index pairs whose padded node rectangles overlap at the given positions. |
| `relaxVertically` | 527 | Iteratively pushes overlapping nodes apart vertically, leaving pinned nodes fixed. |
| `clampToGraph` | 613 | Keeps a node center inside the graph width and the visible viewport height. |
| `positionNodes` | 619 | Computes final node positions for pod or ring mode, applies pins, sets graph height and node styles. |
| `nodeRects` | 870 | Maps each session pid to its padded node rectangle at the current position. |

### storage

| Function | Line | What it does |
|---|---|---|
| `pinKeyOf` | 543 | Returns the stable storage key for a session: label, role, or pid. |
| `removeStoredKeys` | 563 | Deletes every localStorage key starting with the prefix. |
| `folderClosed` | 1394 | Reads whether a task folder is collapsed from localStorage. |

### drag and pin

| Function | Line | What it does |
|---|---|---|
| `pinnedPositionOf` | 547 | Returns a session's pinned position from memory or localStorage, looking up once. |
| `savePin` | 558 | Stores a pinned node position in memory and localStorage. |
| `clearPins` | 571 | Forgets all pinned positions and their stored copies. |
| `resetLayout` | 607 | Clears pins and row order, then re-renders. |
| `node pointer listeners` | 743 | Four delegations wiring pointerdown, move, up, and cancel to the node drag handlers. |
| `pointerInGraph` | 766 | Converts a pointer event to graph-relative coordinates. |
| `startDrag` | 771 | Begins a node drag on primary button, remembering the pointer offset and capturing the pointer. |
| `moveDrag` | 779 | After a small threshold, pins the node at the pointer and schedules a render. |
| `endDrag` | 789 | Ends a node drag, releases capture, saves the pin if moved, and re-renders. |
| `resetLayoutButton click` | 1535 | Wires the reset button to resetLayout. |

### activity rows

| Function | Line | What it does |
|---|---|---|
| `rowRankOf` | 578 | Returns a session's saved row rank from memory or localStorage, looking up once. |
| `saveRowOrder` | 589 | Assigns and persists ranks to the manually ordered row sessions. |
| `clearRowOrder` | 596 | Forgets all manual row ranks and their stored copies. |
| `hasRowOrder` | 603 | True when any live session has a manual row rank. |
| `activityRows` | 1242 | Sorts sessions working first, then most recently seen, then by pid. |
| `lastChange` | 1243 | Returns a session's last_seen_at in ms. |
| `row sort comparator` | 1244 | Inline comparator implementing the automatic row order. |
| `orderedRows` | 1250 | Puts manually ranked rows first in rank order, then the automatic remainder. |
| `rowDragClass` | 1257 | Returns the dragging or drop-indicator class for a row during a row drag. |
| `rowClassName` | 1264 | Composes a row's class list from status, pick, task, and drag state. |
| `rowDropTargetAt` | 1268 | Finds which row the pointer is over and whether to drop above or below it. |
| `placeRow` | 1280 | Moves the dragged row next to the target in DOM order and persists the new ranks. |
| `refreshRowDragClasses` | 1293 | Reapplies row classes so drop indicators follow the pointer. |
| `startRowDrag` | 1298 | Begins a row drag on primary button and captures the pointer. |
| `moveRowDrag` | 1304 | After a threshold, updates the drop target and refreshes row classes. |

| | | |
|---|---|---|
| `endRowDrag` | 1314 | Ends a row drag, places the row unless cancelled, and re-renders. |
| `renderRows` | 1325 | Creates or updates an activity row per session with dot, name, pill, activity, and badge; reorders them. |
| `row click` | 1343 | Picks the session unless the click immediately follows a row drag. |
| `row pointer listeners` | 1344 | Four delegations wiring pointerdown, move, up, and cancel to the row drag handlers. |

### node rendering

| Function | Line | What it does |
|---|---|---|
| `dotEl` | 680 | Creates a colored session dot, with a working halo when requested. |
| `eyeEl` | 686 | Creates the eye emoji span, optionally pulsing. |
| `mailEl` | 704 | Creates the envelope count span with a queued-message tooltip. |
| `buildNodeContent` | 710 | Builds a node's header, status row, and last-activity line for the current mode. |
| `renderNodes` | 731 | Creates or updates a node element per session with classes, content, and listeners; removes stale ones. |
| `node click` | 742 | Picks the session unless the click immediately follows a drag. |
| `hot predicate` | 759 | Inline test for whether an in-flight message touches this session. |
| `waitingText` | 1230 | Returns a waiting-on sentence listing the session's wait targets, or empty. |
| `activityOf` | 1235 | Returns the node's activity line: waiting text, current activity, or empty when idle. |

### tethers

| Function | Line | What it does |
|---|---|---|
| `tetherEyeEl` | 696 | Builds the inline SVG eye icon used in tether glyphs. |
| `tetherOpacity` | 1021 | Computes a tether's opacity from its fade-in and fade-out times. |
| `activeTetherPairs` | 1027 | Maps waiter~target keys to pid pairs from each session's waiting_on list. |
| `trackTethers` | 1037 | Records when tethers appear or vanish so they can fade, dropping finished ones. |
| `curveBetween` | 1049 | Builds a sampled curve between two nodes through a control point with exit gaps. |
| `curveScore` | 1055 | Penalizes a curve for hitting other nodes, crossing curves, or hitting labels. |
| `translatedCurve` | 1065 | Shifts an arrow's curve along a normal and recomputes its end exit point. |
| `shifted` | 1066 | Offsets a point along the normal by the given distance. |
| `curveAtDistance` | 1072 | Translates a curve and corrects the shift so its midpoint lands at the target distance. |

| | | |
|---|---|---|
| `besideArrow` | 1080 | Chooses a tether curve running beside an existing arrow, or a free curve if that scores better. |
| `side forEach` | 1094 | Inline loop trying preferred then opposite side at three step distances. |
| `placeGlyph` | 1109 | Picks the least-penalized position along a tether curve for the eye glyph. |
| `podNeedsLowerLane` | 1139 | In pod mode, true when arrows or tethers need a second lane below. |
| `renderTethers` | 1150 | Places each tracked tether curve and glyph, hides crossed labels, and creates or updates their elements. |
| `tether sort` | 1157 | Inline comparator ordering tethers by key. |
| `sameWay and otherWay find` | 1164 | Inline lookups for an arrow in the same or opposite direction as the tether. |

### labels and overlap avoidance

| Function | Line | What it does |
|---|---|---|
| `stageSpan` | 811 | Creates a span with the stage label text and class. |
| `labelSize` | 879 | Estimates an arrow label's width and height from summary and stage text. |
| `placeArrow` | 897 | Scores every control point, label position, and side; returns the lowest-penalty curve and label. |
| `controlCandidates forEach` | 901 | Inline loop scoring one control point against nodes, other curves, and label slots. |
| `placeAll` | 929 | Places all arrows over two rounds against each other, then flags labels crossed by others. |
| `others map` | 933 | Inline projection of already placed fits into points plus optional label rect. |
| `crossedByOther` | 939 | Inline test for whether any other arrow curve hits this label. |
| `hideCrossedLabel` | 1129 | Marks an arrow label unclear and hides it unless lit. |

### geometry

| Function | Line | What it does |
|---|---|---|
| `bezierPoint` | 816 | Evaluates a quadratic bezier at parameter t. |
| `bezierNormal` | 821 | Returns the unit normal of a quadratic bezier at parameter t. |
| `sampleCurve` | 828 | Samples a quadratic bezier into a fixed number of points. |
| `rectAround` | 834 | Builds a rectangle of given size centered on a point. |
| `rectsOverlap` | 838 | True when two rectangles intersect. |
| `pointInRect` | 842 | True when a point lies inside a rectangle. |
| `orientation` | 846 | Cross-product sign test for three points. |
| `segmentsCross` | 850 | True when two line segments properly intersect. |
| `curveHitsRect` | 855 | True when a sampled curve enters or crosses a rectangle. |

| | | |
|---|---|---|
| `curvesCross` | 864 | Counts segment intersections between two sampled curves. |

### strip and pick

| Function | Line | What it does |
|---|---|---|
| `renderStrip` | 1211 | Rebuilds the chip strip with a clickable chip per session showing status and inbox count. |
| `chip click` | 1220 | Picks the chip's session. |
| `pick` | 1225 | Toggles the picked session and re-renders. |

### render loop

| Function | Line | What it does |
|---|---|---|
| `render` | 1468 | Full frame: layout, nodes, arrows, tethers, caption, reset button, strip, rows, panel, title. |
| `visible in-layout filter` | 1481 | Inline filter keeping arrows whose both endpoints have positions. |
| `scheduleRender` | 1493 | Coalesces render requests into one deferred render. |
| `scheduleRender timeout` | 1496 | Clears the queued flag and runs render. |
| `window resize` | 1536 | Schedules a render on resize. |
| `render interval` | 1537 | Re-renders every 250 ms so fades and pulses advance. |

### SSE handling

| Function | Line | What it does |
|---|---|---|
| `connectEvents` | 1512 | Opens the EventSource and wires open, message, and error handlers. |
| `source.onopen` | 1514 | Shows live status and stops the polling fallback. |
| `source.onmessage` | 1519 | Parses a change event, applies it, and schedules a render; shows parse errors. |
| `source.onerror` | 1527 | Shows reconnecting status and starts polling the state endpoint every two seconds. |

### Suggested split

| New module | Purpose | Takes these groups |
|---|---|---|
| `web.css` | All page styling moved out of the inline style block. | css |
| `web-state.js` | Constants, DOM handles, state record, feed clock, snapshot and change application, fetch and SSE connection. | state and fetch, SSE handling, utilities, storage |
| `web-messages.js` | Pure queries over messages: endpoints, stages, visibility, read fading, idle pulses, status text. | message queries, caption |
| `web-geometry.js` | Bezier, rectangle, and segment math shared by arrows and tethers. | geometry |

| | | |
|---|---|---|
| `web-layout.js` | Ring and pod layout, node sizing, overlap relaxation, pins, node drag. | ring layout and geometry, drag and pin |
| `web-arrows.js` | Arrow placement and label scoring, tether placement, glyphs, and their DOM. | arrows and curves, labels and overlap avoidance, tethers |
| `web-panels.js` | Node content, chip strip, activity rows with row drag, task panel, title. | node rendering, strip and pick, activity rows, task panel |
| `web-main.js` | The render frame, scheduleRender, intervals, and top-level listener wiring. | render loop |

**Observations**

- CSS is 120 lines (lines 9-128) and is independent of the script, so it can move to web.css with a single link tag.
- The script is 1390 lines in one scope; modules share state through closure globals (layout, pins, nodeEls, pickedPid, litKey), so splitting into files needs either ES modules with explicit exports or an ordered sequence of classic scripts sharing globals.
- renderArrows (945-1019) and renderTethers (1150-1209) are the longest functions at roughly 70 and 60 lines and mix placement, scoring, and DOM updates.
- placeArrow (897-927) and placeGlyph (1109-1127) repeat the same label-slot scoring loop: bounds penalty, node rect overlap, placed label overlap, curve hits.
- placeArrow's inner curve scoring (904-909) duplicates curveScore (1055-1063) line for line.
- The free-curve fallback in besideArrow (1103) and renderTethers (1171) build the same placeArrow entry with GLYPH_SIZE and [0] side offsets.
- Node drag (766-799) and row drag (1298-1323) follow the same pointer capture, threshold, and click-after-drag suppression pattern with separate state variables.
- Pin storage (543-576) and row rank storage (578-605) are parallel memory-plus-localStorage caches with the same once-only lookup pattern.
- renderNodes (731-764) and renderRows (1325-1362) both recompute taskedPids and statusText and build the same dot, name, pill, badge content.
- Click-after-drag suppression in node click (742) and row click (1343) is identical apart from the lastDragEnd variable.
- The label element width measured in renderArrows (1011) feeds back into fit.labelRect that renderTethers later reads, coupling the two renderers through mutation.
- Dim and lit computations for arrows (992, 995) and tethers (1188, 1201) and hideCrossedLabel (1135) each re-derive picked-pid involvement.
- render (1468-1490) reads layout.pinned which is set as an extra property on the layout object by positionNodes rather than declared in layoutNodes.
- feedClock, applySnapshot, and applyChange are the only places that touch raw feed timestamps, so they form a clean state module boundary.

## 7. `viz/index.html` 638 lines, 51 functions

Inline CSS: about 145 lines.

### utilities

| Function | Line | What it does |
|---|---|---|
| `folderOf` | 186 | Returns the last path segment of a working directory or (unknown). |
| `truncate` | 191 | Collapses whitespace and trims text to a maximum length with an ellipsis. |
| `labelOf` | 196 | Picks a display label for a session: label, role, name, or pid. |
| `labelFor` | 200 | Resolves a pid to a session label, falling back to the raw value. |
| `pidByName` | 346 | Finds a session pid whose name or label matches the given string. |
| `stageText` | 389 | Returns the human stage label for a message. |
| `svgEl` | 431 | Creates an SVG element with the given attributes. |

### card rendering

| Function | Line | What it does |
|---|---|---|
| `dotColor` | 206 | Chooses a dot color for manager or builder roles, else a palette color by pid order. |
| `lastLineOf` | 286 | Picks the card's quoted last line from the newer of latest send or latest read. |
| `pill` | 294 | Creates a pill span with text and class. |
| `buildCard` | 301 | Builds a session card with dot, label, status and inbox pills, watching line, and last line. |

### state and fetch

| Function | Line | What it does |
|---|---|---|
| `applySnapshot` | 214 | Replaces all state from a snapshot and shows any idle notices still within their lifetime. |
| `applyChange` | 232 | Applies one SSE change event to sessions, messages, sources, and generatedAt. |
| `loadState` | 596 | Fetches the full state snapshot and schedules a render, even on failure. |

### notices

| Function | Line | What it does |
|---|---|---|
| `showNotice` | 226 | Marks an idle notice live and schedules its removal and a render after four seconds. |
| `showNotice timeout` | 229 | Drops the expired notice and schedules a render. |
| `liveNoticeFor` | 380 | Finds a live idle notice whose endpoints match the given pair. |

### message queries

| Function | Line | What it does |
|---|---|---|
| `messagesOnly` | 254 | Returns stored records of kind message. |

| | | |
|---|---|---|
| `latestSendOf` | 258 | Finds the most recent message sent by a pid. |
| `latestReadBy` | 267 | Finds the most recently read message addressed to a pid. |
| `isSending` | 276 | True when a working session's latest send is still pending. |
| `statusText` | 281 | Returns Idle, Sending, or Working for a session. |
| `endpointsOf` | 351 | Returns [fromPid, toPid] for a message, resolving a missing to_pid by name. |
| `messageTime` | 358 | Returns the message's latest lifecycle timestamp string. |
| `pairKeyOf` | 362 | Returns an order-independent key for two pids. |
| `messagesByPair` | 366 | Groups messages by unordered pid pair, each list sorted chronologically. |
| `pair sort comparator` | 376 | Inline comparator ordering a pair's messages by lifecycle time then send time. |

### link rendering

| Function | Line | What it does |
|---|---|---|
| `buildLink` | 393 | Builds the between-card link showing the latest message summary, arrow, stage, and any notice. |
| `link click` | 419 | Opens the history panel for this pair. |

### arcs

| Function | Line | What it does |
|---|---|---|
| `topCenterOf` | 437 | Returns the top-center point of a session card relative to the arcs origin. |
| `drawArcs` | 444 | Draws a tiered quadratic arc with summary and stage text for each non-adjacent pair. |
| `arc text click` | 473 | Opens the history panel for the arc's pair. |
| `rowArcs sort` | 571 | Inline comparator ordering arcs by span then left index for tiering. |
| `tier assign` | 572 | Inline loop giving each arc its tier index. |

### history panel

| Function | Line | What it does |
|---|---|---|
| `openPanel` | 479 | Records the open pair, slides the panel in, and renders it. |
| `closePanel` | 485 | Clears the open pair and slides the panel out. |
| `renderPanel` | 490 | Lists the open pair's messages newest first with stage, summary, body, and times. |
| `times map` | 510 | Inline formatter printing sent, accepted, enqueued, and read clock times. |
| `panel-close click` | 516 | Wires the close button to closePanel. |

### render loop

| Function | Line | What it does |
|---|---|---|
| `window resize` | 517 | Schedules a render on resize. |
| `render` | 528 | Rebuilds header counts and every folder row with cards, links, tiered arcs, and caption. |
| `folders sort` | 550 | Inline default sort of folder entries by name. |
| `ordered sort` | 560 | Inline comparator ordering a folder's sessions by pid. |
| `ordered forEach` | 561 | Inline loop appending a card and a link to the next card. |
| `scheduleRender` | 590 | Coalesces render requests into one deferred render. |
| `scheduleRender timeout` | 593 | Clears the queued flag and runs render. |

### caption

| Function | Line | What it does |
|---|---|---|
| `latestSourceAmong` | 519 | Returns the source text of the highest-sequence source among the given pids. |

### SSE handling

| Function | Line | What it does |
|---|---|---|
| `connectEvents` | 610 | Opens the EventSource and wires open, message, and error handlers. |
| `source.onopen` | 612 | Shows live status with the event count and stops the polling fallback. |
| `source.onmessage` | 617 | Counts the event, applies the parsed change, schedules a render; shows parse errors. |
| `source.onerror` | 627 | Shows reconnecting status and starts polling the state endpoint every two seconds. |

### Suggested split

| New module | Purpose | Takes these groups |
|---|---|---|
| `index.css` | All page styling moved out of the inline style block. | css |
| `index-state.js` | State record, snapshot and change application, notices, fetch and SSE. | state and fetch, SSE handling, notices, utilities, message queries |
| `index-board.js` | Cards, links, arcs, history panel, caption, and the render loop. | card rendering, link rendering, arcs, history panel, caption, render loop |

### Observations

- CSS is 145 lines (lines 8-152), larger than web.html's, and can move to index.css unchanged.
- The script is 471 lines; render (528-587) is the longest function and does grouping, card and link building, arc tiering, and caption in one pass.
- folderOf, truncate, labelOf, labelFor, pidByName, endpointsOf, messagesOnly, latestSendOf, statusText, svgEl, applySnapshot, applyChange, scheduleRender, loadState, and connectEvents are near-copies of the same functions in web.html and could be one shared file.

- buildLink (393-429) and drawArcs (444-477) both render the latest message of a pair with summary, arrow direction, stage, and live notice in different DOM.
- render replaces the whole board each frame, unlike web.html's keyed element reuse, so card positions are remeasured for arcs after every render.
- The resize listener and panel-close wiring (516-517) sit between function definitions rather than with the other top-level wiring at 634-635.